

BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY)

COLLEGE OF ENGINEERING, PUNE – 411043

A PROJECT BASED LEARNING

REPORT ON

HOSTELFIX

**SMART HOSTEL MAINTENANCE ANALYTICS
& PRIORITY PREDICTION SYSTEM**

Submitted by

Name	PRN	Roll No.
Pankaj Singh	2414110297	90

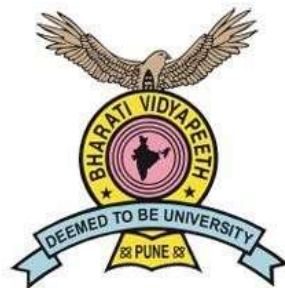
For the course: Data Warehousing and Mining

Submitted To Course Coordinator

Dr. Rohini Khalkar

B. Tech Computer Engineering (Sem V)

2026–27



DEPARTMENT OF COMPUTER ENGINEERING

Bharati Vidyapeeth (Deemed to be University)

College of Engineering, Pune – 411043



CERTIFICATE

This is to certify that the project report entitled “HostelFix: Smart Hostel Maintenance Analytics & Priority Prediction System” is a bonafide work carried out by Pankaj Singh (PRN 2414110297, Roll No. 90) in partial fulfilment of the requirements for the award of the degree of B. Tech in Computer Engineering during the academic year 2026–27. The work has been completed under my guidance and supervision. To the best of my knowledge, the work presented in this report has not been submitted, either in whole or in part, to any other university or institution for the award of any degree or diploma.

The report describes a student prototype that applies data preprocessing and data-mining techniques to hostel maintenance complaint records. Predictions are decision-support estimates and should be reviewed by hostel staff before action.

Date:

Dr. Rohini Khalkar

Internal Guide

Dept. of Computer Engineering

Dr. Sandeep Vanjale

Head of Department

Dept. of Computer Engineering

ACKNOWLEDGEMENT

I express my sincere gratitude to Dr. Rajesh Prasad, Principal of Bharati Vidyapeeth (Deemed to be University), College of Engineering, Pune, for providing the academic environment and facilities required for completing this project.

I am deeply thankful to Dr. Sandeep Vanjale, Head of the Department of Computer Engineering, for encouragement and support throughout the project work. I extend my heartfelt gratitude to my project guide, Dr. Rohini Khalkar, for continuous guidance, constructive suggestions, and motivation during the preparation of this report.

I also thank all faculty members, friends, and family members who supported the learning process. This project allowed me to connect Data Warehousing and Data Mining concepts—preprocessing, classification, regression, clustering, and association-rule mining—with a practical hostel maintenance scenario. Finally, I thank my institution for providing an opportunity to undertake this project.

Pankaj Singh

TABLE OF CONTENTS

CERTIFICATE	2
ACKNOWLEDGEMENT	3
TABLE OF CONTENTS	4
ABSTRACT	7
1. INTRODUCTION	8
1.1 Background	8
1.2 Problem Statement	8
1.3 Objectives	8
1.4 Scope	9
1.5 Need for the System	9
1.6 Organization of the Report	9
2. LITERATURE SURVEY	10
2.1 Introduction	10
2.2 Classification	10
2.3 Regression	10
2.4 Clustering	10
2.5 Association Rules	10
2.6 Evaluation of Models	11
2.7 Comparison of Approaches	11
2.8 Research Gap	11
3. REQUIREMENT ANALYSIS AND FEASIBILITY STUDY	12
3.1 Existing System	12
3.2 Proposed System	12
3.3 Functional Requirements	12
3.4 Non-Functional Requirements	13
3.5 Feasibility Study	13
3.6 Use-Case Description	13

4. SYSTEM DESIGN	14
4.1 System Architecture	14
4.2 Main Components	14
4.3 Data-Flow Description	14
4.4 Proposed Complaint Data Model	15
4.5 Database Design	16
4.6 Dashboard Design	17
5. METHODOLOGY AND IMPLEMENTATION	19
5.1 Development Methodology	19
5.2 Data Collection	19
5.3 Data Preprocessing	19
5.4 Importance of Data Quality	19
5.5 Model Training and Evaluation Workflow	20
5.6 Decision Tree	20
5.7 Naive Bayes	21
5.8 Linear Regression	21
5.9 K-Means	22
5.10 Apriori Association Rules	22
5.11 Modules of the System	23
5.12 Technology Stack	24
5.13 Feature List	24
5.14 Implementation Details	24
6. VISUAL ANALYTICS AND RESULTS	26
6.1 Visual Analytics	26
6.2 Evaluation Metrics	27
6.3 Recommended Test Cases	28
6.4 Results and Discussion	28
7. CONCLUSION AND FUTURE SCOPE	30

7.1 Conclusion	30
7.2 Expected Benefits	30
7.3 Limitations	30
7.4 Future Scope	30
REFERENCES	31

ABSTRACT

Hostel maintenance complaints about water leakage, electrical faults, Wi-Fi problems, cleaning, and damaged furniture are often recorded in registers or passed on through messages. Such scattered records are hard to search, make recurring problems difficult to notice, and leave urgent work without a clear order of priority. HostelFix is a web-based concept that organizes these complaint records and uses data-mining methods to support faster, better-informed decisions by hostel staff.

The prototype is designed around Python and Flask with SQLite, HTML, CSS, JavaScript, Pandas, NumPy, and scikit-learn. A Decision Tree and a Naive Bayes classifier can be compared for priority classification (Low, Medium, High, Critical); Linear Regression can estimate the number of days needed to resolve a complaint; K-Means can group similar complaints; and Apriori can identify frequent co-occurrences such as a complaint category that repeatedly appears in the same hostel block. A dashboard presents complaint status and trends, and staff remain responsible for reviewing every estimate before acting on it.

This report explains the requirements, architecture, data model, preprocessing, algorithms, visual analytics, testing plans, benefits, and limitations of the system. The source case study uses sample or synthetic training data, so the example charts are clearly marked as illustrative rather than actual project measurements, and the test cases are recommended verification cases rather than executed results.

Keywords: hostel maintenance, data mining, preprocessing, classification, regression, clustering, association rules, Flask, SQLite.

1. INTRODUCTION

1.1 Background

Hostels receive maintenance requests involving plumbing, electrical systems, Wi-Fi, cleaning, and furniture. When reports are scattered across registers, phone calls, or messages, staff may find it difficult to search older records, identify repeated problems, and decide which job should be done first. Structured data and basic analytics can make this process easier to review.

Data mining is the process of discovering useful patterns in stored data. Typical tasks include classification (predicting a category), regression (predicting a number), clustering (grouping similar records), and association-rule mining (finding items that occur together) [1][2]. A hostel complaint database is a small but realistic setting in which all four tasks are meaningful, which makes it a suitable case study for the Data Warehousing and Mining course.

1.2 Problem Statement

The problem addressed by HostelFix is the lack of a unified, searchable record of maintenance complaints and their trends. Manual tracking can delay follow-up and hide recurring issues. Staff also have no consistent, data-based way to judge which complaints are most urgent or how long a repair is likely to take.

The central problem addressed by this project is: How can a web application record hostel maintenance complaints and use data-mining methods to estimate priority, estimate resolution time, and reveal recurring patterns, so that staff can review the estimates and make better decisions? The system is meant to assist—not replace—staff judgment.

1.3 Objectives

- To record, search, and update maintenance complaints in a structured form.
- To estimate priority classes such as Low, Medium, High, and Critical using classification models.
- To estimate approximate resolution duration using regression.
- To explore groups and recurring combinations in complaints using clustering and association rules.
- To display complaint status and trends through a dashboard.
- To describe the preprocessing and evaluation steps needed for reliable results.
- To demonstrate Data Warehousing and Data Mining concepts in a practical hostel setting.

1.4 Scope

The system is an educational prototype for hostel complaint management and analytics. It is intended to demonstrate how complaint records can be organized and analysed. Predictions depend on the size and quality of the dataset. A real deployment would need actual labelled hostel records, secure access, persistent storage, and systematic model evaluation.

Within scope are complaint entry, validation, storage, priority and resolution-time estimation, pattern exploration, and a dashboard. Outside the scope of this report are automatic dispatch of repair staff, billing, inventory of spare parts, integration with hostel-entry systems, and mobile applications.

1.5 Need for the System

Delays in repairing leaks, electrical faults, or network outages affect the comfort, safety, and study time of many students. A system that keeps all complaints in one place, highlights the potentially urgent ones, and shows which problems keep coming back helps staff plan their work and helps students see that their reports are tracked. It also shows how data-mining techniques are applied to an everyday problem.

1.6 Organization of the Report

Chapter 1 introduces the project. Chapter 2 presents the literature survey. Chapter 3 discusses requirements and feasibility. Chapter 4 explains system design. Chapter 5 describes methodology and implementation. Chapter 6 presents visual analytics, evaluation measures, and test cases. Chapter 7 concludes the project and outlines limitations and future enhancements.

2. LITERATURE SURVEY

2.1 Introduction

This chapter reviews the data-mining concepts used by HostelFix: classification, regression, clustering, association rules, and the evaluation of models. The descriptions follow standard textbooks and library documentation [1][2][3].

2.2 Classification

Classification learns from labelled examples and predicts a category for a new record. A Decision Tree builds understandable feature-based branches by repeatedly choosing the split that best separates the classes [10]. Naive Bayes applies Bayes' theorem with the assumption that features are conditionally independent given the class, and estimates a probability for every class [1][2]. Both models can predict complaint priority and should be compared on the same test records.

2.3 Regression

Regression predicts a numerical output. HostelFix can use Linear Regression to estimate the number of days needed to resolve a complaint, based on available details such as complaint type, severity, pending days, and affected rooms. It is simple and fast, but it assumes that the relationship between the inputs and the output is roughly linear.

2.4 Clustering

K-Means groups records with similar feature patterns without needing a known class for every record [11]. Clusters can help staff investigate whether some complaint types share similar severity or waiting-time patterns. Because K-Means uses distances, features measured on different scales usually need to be scaled first.

2.5 Association Rules

Apriori identifies attributes that frequently occur together [9]. Support measures the frequency of an item combination, confidence measures the frequency of the consequent when the antecedent occurs, and lift compares the rule with the consequent's baseline frequency. A rule is interesting only if it has enough support, a useful confidence, and a lift above 1.

2.6 Evaluation of Models

For classification, accuracy, precision, recall, and F1-score give a more complete comparison than accuracy alone, especially when some priority classes are rare. For regression, the mean absolute error (MAE) reports the average error in days. When data are scarce, k-fold cross-validation gives a more stable estimate than a single small hold-out split [1].

2.7 Comparison of Approaches

Table 1: Comparison of Approaches Used in HostelFix

Approach	Purpose	Strength	Limitation
Decision Tree	Priority	Explainable branches	May overfit
Naive Bayes	Priority	Fast probability model	Simplifying assumptions
Linear Regression	Resolution days	Simple numeric estimate	May miss nonlinear effects
K-Means	Grouping	Explores patterns	Needs sensible scaling
Apriori	Associations	Readable rules	May generate weak rules

2.8 Research Gap

Many student maintenance projects stop at a form and a table of complaints. This project combines complaint management with four different data-mining tasks on the same records, and it states clearly which results are illustrative and which would have to be measured on a real dataset, so that example figures are never mistaken for project findings.

3. REQUIREMENT ANALYSIS AND FEASIBILITY STUDY

3.1 Existing System

A manual process may rely on conversations, phone calls, paper registers, or messages. This can scatter information. A student may not know whether a complaint was received, staff cannot easily see how many similar complaints are pending, and there is no record that can be searched or summarized when a problem repeats.

3.2 Proposed System

HostelFix proposes a common complaint form and a dashboard so that records can be searched and grouped. The system validates and stores each complaint, estimates its priority and likely resolution time using trained models, explores recurring patterns, and lets staff review the estimates and update complaint status.

3.3 Functional Requirements

Table 2: Functional Requirements

ID	Functional requirement
FR-01	The system shall accept complaint category, location, description, and severity.
FR-02	The system shall validate required fields and flag incomplete entries.
FR-03	The system shall store complaint records in the prototype database.
FR-04	The system shall estimate priority using a classification model.
FR-05	The system shall estimate resolution duration using regression.
FR-06	The system shall display complaint lists, status, and basic charts.
FR-07	The system shall allow exploration of similar records and frequent associations.
FR-08	The system shall allow staff to review and update complaint status.
FR-09	The system shall allow complaints to be searched and filtered by category, block, and status.
FR-10	The system shall reject invalid input with a clear message.

3.4 Non-Functional Requirements

Table 3: Non-Functional Requirements

ID	Non-functional requirement
NFR-01	Usability: forms and charts should be understandable for students and staff.
NFR-02	Reliability: records should be validated and stored consistently.
NFR-03	Performance: routine views and predictions should respond promptly.
NFR-04	Security: limit personal data and control access.
NFR-05	Maintainability: keep routes, models, storage, and UI modules separate.
NFR-06	Transparency: model estimates must be shown as estimates for staff review, never as final decisions.

3.5 Feasibility Study

Technical feasibility: Python, Flask, SQLite, and scikit-learn provide everything needed for storage, model training, prediction, and a web interface in a student prototype.

Economic feasibility: the main tools are open source and free, so the prototype can be built and run at no cost.

Operational feasibility: a simple form can fit hostel workflows; students describe the problem and staff read the dashboard.

Schedule feasibility: development can proceed in stages: data storage, preprocessing, models, dashboard, testing, and documentation.

3.6 Use-Case Description

The primary actors are the student and the maintenance staff member. The student opens the complaint form, enters the category, location, description, and severity, and submits it. The system validates and stores the record and returns a confirmation. Staff open the dashboard, review the list of complaints with their estimated priority and resolution time, and update the status as work progresses. A developer or administrator may retrain the models when new, well-labelled records become available.

4. SYSTEM DESIGN

4.1 System Architecture

The architecture separates the web interface, application logic, data storage, preprocessing, and analytical models. The browser collects complaint details. Flask validates requests and connects the interface to storage and prediction functions. SQLite stores prototype records, while data-processing code prepares features for analytics.

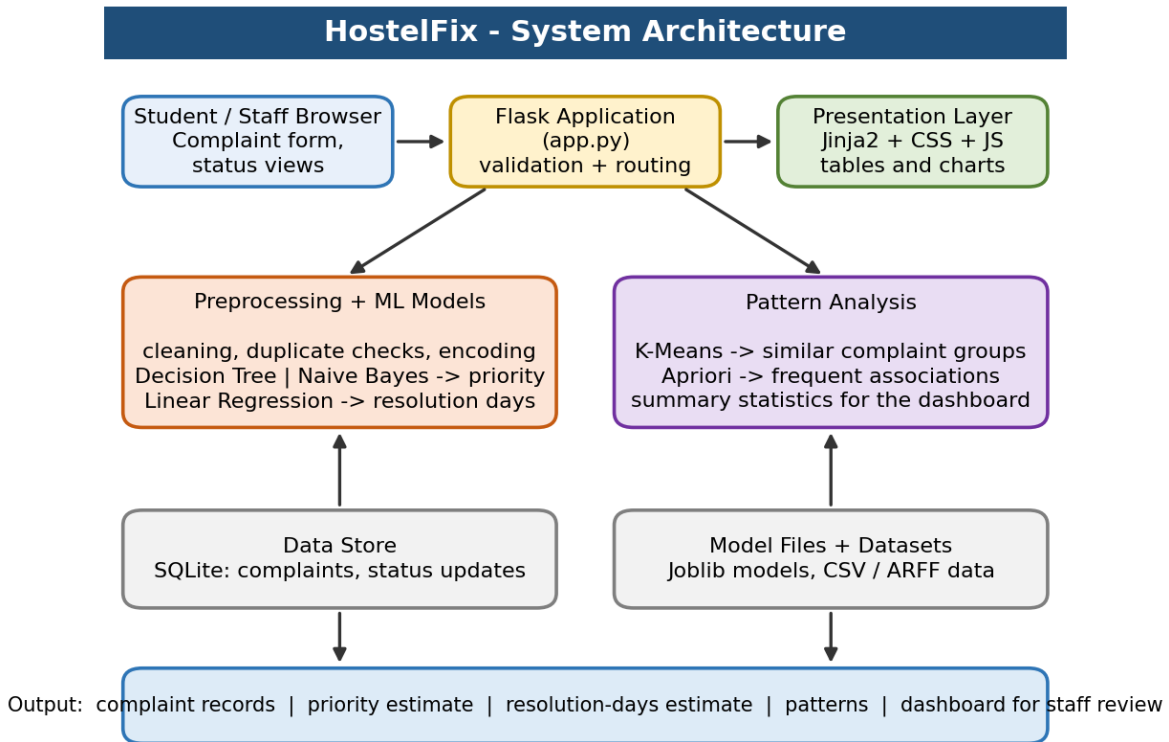


Figure 4.1: System Architecture

4.2 Main Components

- **Presentation layer:** complaint form, tables, status views, and charts.
- **Application layer:** request validation, routing, and module coordination.
- **Data layer:** SQLite tables for complaints and updates.
- **Preprocessing layer:** cleaning, duplicate checks, encoding, and feature preparation.
- **Analytics layer:** classification, regression, clustering, and association rules.

4.3 Data-Flow Description

- 1 The student submits a complaint through the form.

- 2 The application checks that required fields are present and valid.
- 3 The record is stored in the SQLite database.
- 4 Preprocessing prepares the relevant fields (cleaning, encoding, scaling where needed).
- 5 Classification models estimate priority; regression estimates resolution days.
- 6 Clustering and association rules explore recurring patterns across stored records.
- 7 The dashboard presents the information for staff review. Staff remain responsible for decisions and status updates.

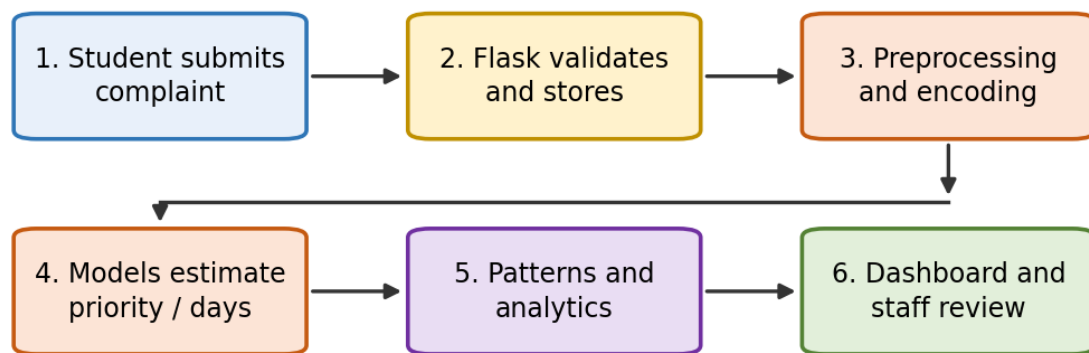


Figure 4.2: Conceptual workflow of HostelFix

4.4 Proposed Complaint Data Model

The proposed fields below are used by the models. The actual dataset schema should be checked before implementation or evaluation.

Table 4: Proposed Complaint Data Model

Field	Description	Example	Role
Complaint_ID	Unique identifier	HF-0001	Identifier
Category	Maintenance issue	Water leakage	Input (encoded)
Block / Location	Issue location	Block B, bathroom	Input / grouping
Severity	Reported impact	High	Input (ordinal)
Days_Pending	Waiting duration	2	Input (numeric)
Affected_Rooms	Rooms impacted	3	Input (numeric)
Previous_Complaints	Earlier similar reports	2	Input (numeric)
Priority	Classification target	High	Target (class)
Resolution_Days	Regression target	3	Target (number)

Priority is treated as one of four classes. The following guideline is proposed so that labels are applied consistently; it should be agreed with hostel staff before real data are labelled.

Table 5: Proposed Priority Classes

Class	Meaning (proposed guideline)	Example
Low	Minor inconvenience with no safety or hygiene impact	Loose chair
Medium	Affects the comfort or study of one student or room	Slow Wi-Fi in one room
High	Affects several rooms or an essential service	Bathroom water leakage
Critical	Safety or health risk needing immediate attention	Exposed live wiring

4.5 Database Design

There is no large database in the prototype. SQLite stores complaint records and the history of status updates, and trained models are saved as files. Figure 4.3 shows the proposed entities and Table 6 lists their columns.

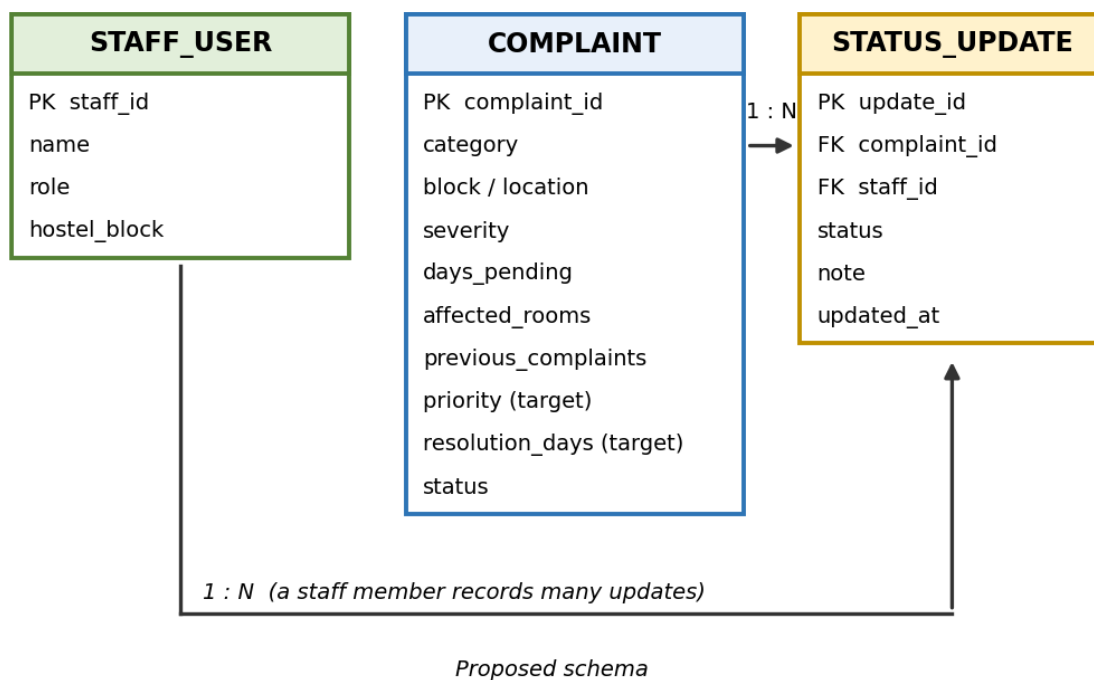


Figure 4.3: Proposed entity-relationship view

Table 6: Proposed Database Tables

Table	Main columns	Purpose
complaints	complaint_id (PK), category, block, location, description, severity, days_pending, affected_rooms, previous_complaints, priority_estimate, priority_final, resolution_estimate, status, created_at	Stores each complaint with model estimates and the staff-confirmed priority
status_updates	update_id (PK), complaint_id (FK), staff_id (FK), status, note, updated_at	Keeps the history of status changes for each complaint
staff_users	staff_id (PK), name, role, hostel_block	Identifies who updated a complaint

4.6 Dashboard Design

Useful dashboard cards include total open complaints, high-priority complaints, average waiting time, and resolved complaints. Charts can compare categories, show resolution-time distributions, and summarize patterns by hostel block. Each statistic should be derived from stored records and display the time period covered.

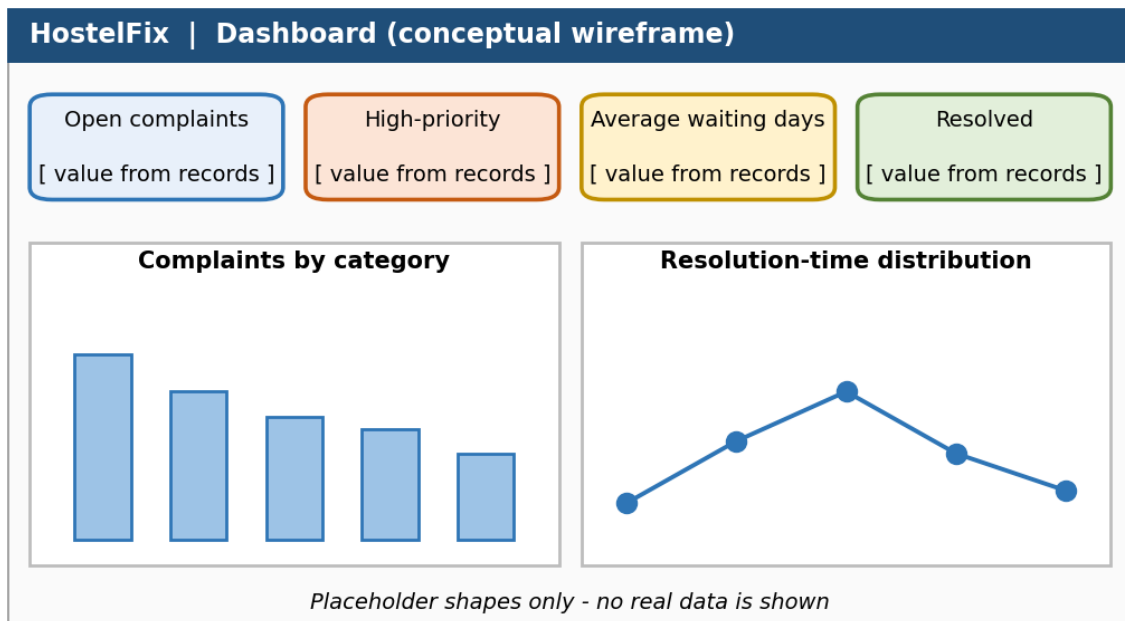


Figure 4.4: Conceptual dashboard wireframe (placeholders, no real data)

Table 7: Proposed Dashboard Components

Component	Source	Purpose
Open complaints card	Count of records with status Open or In progress	Show current workload
High-priority card	Records with High or Critical priority	Draw attention to urgent work
Average waiting time	Mean of days pending for open records	Monitor delays
Resolved card	Records with status Resolved	Show completed work
Category chart	Group by category	Compare complaint volumes
Resolution-time chart	Distribution of resolution days	Show typical repair duration
Block summary	Group by hostel block	Find locations with repeated issues

5. METHODOLOGY AND IMPLEMENTATION

5.1 Development Methodology

Development can proceed incrementally. Each stage is verified before the next one starts.

- 1 Define the fields and storage.
- 2 Validate and preprocess records.
- 3 Train and evaluate the models.
- 4 Connect the models to the Flask application.
- 5 Build the dashboard views.
- 6 Test the complete workflow and document the results.

5.2 Data Collection

Training data may come from historical maintenance registers or a carefully labelled sample. Useful fields include category, location, severity, pending days, affected rooms, final priority, and actual resolution duration. Synthetic data can demonstrate the pipeline but cannot establish real-world performance. The source case study uses sample or synthetic data, so no accuracy figure is claimed in this report.

5.3 Data Preprocessing

- **Cleaning:** identify missing, inconsistent, or invalid values.
- **Duplicate handling:** identify repeated entries for the same incident.
- **Encoding:** convert categories into model-friendly representations.
- **Feature selection:** retain fields relevant to the target.
- **Scaling:** prepare numeric fields when required, especially for distance-based clustering.
- **Train/test separation:** evaluate on records not used to fit the model.

5.4 Importance of Data Quality

If category labels such as “water leak” and “water leakage” are treated as unrelated, counts become fragmented. Missing severity should be handled with a documented method rather than guessed. Data leakage must also be avoided by separating training and testing records correctly: for example, a feature that is only known after a complaint is resolved must not be used to predict its priority.

5.5 Model Training and Evaluation Workflow

The records are split into training and test sets with a fixed random seed so that results can be repeated. Encoders and scalers are fitted on the training records only. The Decision Tree and Naive Bayes models are trained on the same features and compared on the same test records; Linear Regression is trained on the numeric resolution-days target. Because a small dataset gives an unstable single split, k-fold cross-validation is recommended. Trained models are saved with Joblib so that the web application can load them without retraining.

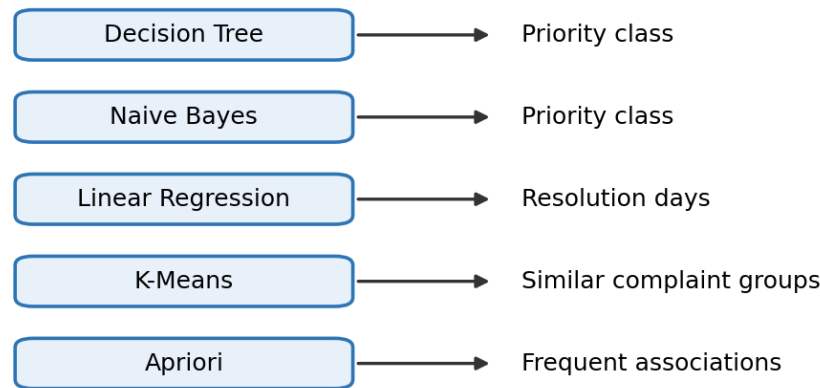


Figure 5.1: Algorithm-to-task map

5.6 Decision Tree

A Decision Tree predicts priority through feature-based branches and is relatively easy to explain. At each node the algorithm chooses the feature and threshold that make the resulting groups purer. One common impurity measure is the Gini index:

$$Gini(t) = 1 - \sum p_i^2$$

where p_i is the fraction of records in node t that belong to class i . For example, if a hypothetical node contains 10 records with 4 High, 3 Medium, 2 Low, and 1 Critical, then $Gini = 1 - (0.16 + 0.09 + 0.04 + 0.01) = 0.70$. A pure node has $Gini = 0$. A tree that is allowed to grow too deep may memorize the training data, so the depth should be limited and checked on test records.

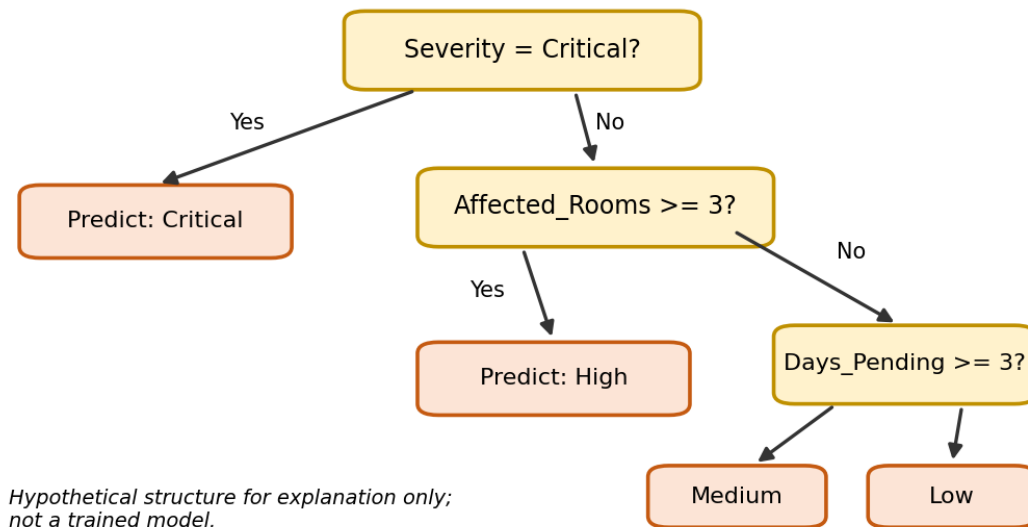


Figure 5.2: Illustrative decision-tree structure (hypothetical, not trained)

5.7 Naive Bayes

Naive Bayes predicts using probabilities based on input features. For a record with features $x_1, \dots, x_n$, the score of each priority class C is:

$$P(C | x_1, \dots, x_n) \text{ is proportional to } P(C) \times \prod P(x_i | C)$$

The class with the highest score is predicted. The “naive” assumption that features are independent is rarely perfectly true (for example, severity and affected rooms may be related), but the model is fast, needs little data, and also gives a probability for every class, which helps staff judge how certain an estimate is. The Decision Tree and Naive Bayes models should use the same features and test set. Accuracy, precision, recall, and F1-score provide a more complete comparison than accuracy alone.

5.8 Linear Regression

Linear Regression returns a numeric estimate, such as expected resolution days:

$$\text{Resolution_Days} = b_0 + b_1 \times \text{Severity_Score} + b_2 \times \text{Days_Pending} + b_3 \times \text{Affected_Rooms} + \dots$$

The coefficients are chosen to minimise the sum of squared differences between the actual and predicted days on the training records. Its usefulness depends on the quality of historical resolution records and on whether a linear relationship is reasonable for the selected features. Predictions below zero days are meaningless and should be limited to a sensible minimum when shown to staff.

5.9 K-Means

K-Means groups records based on similarity across selected features. It places K centres, assigns every record to its nearest centre, recomputes each centre as the mean of its records, and repeats until the groups stop changing. It tries to minimise:

$$J = \sum_k \sum_{x \text{ in cluster } k} \|x - \mu_k\|^2$$

Features measured on different scales need scaling first; otherwise a feature with large values, such as the number of days, would dominate one with small values. Min-max scaling uses $x' = (x - \min) / (\max - \min)$. The number of clusters K must be chosen sensibly and the groups should be interpreted by staff, because a cluster is only a pattern and not a diagnosis.

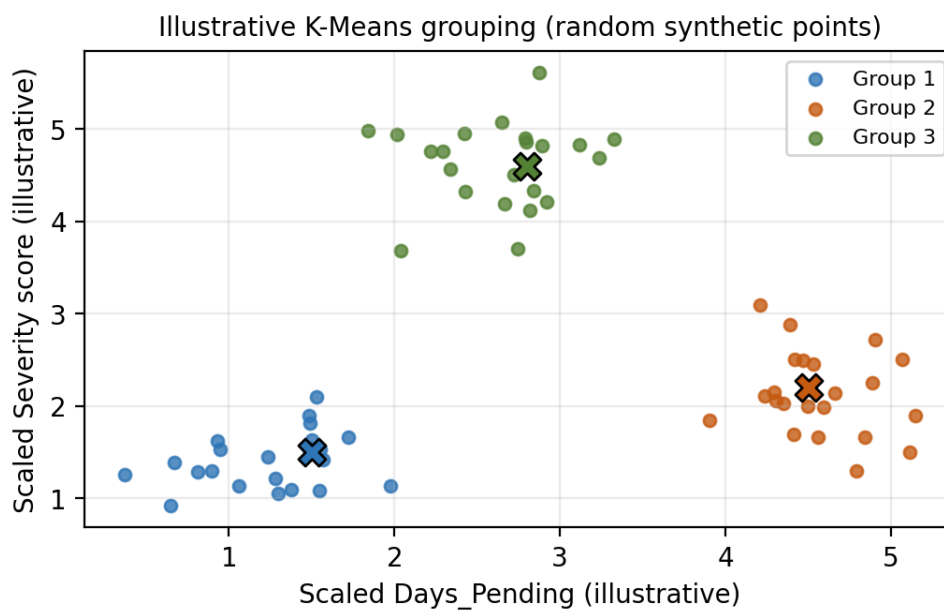


Figure 5.3: Illustrative K-Means grouping (random synthetic points)

5.10 Apriori Association Rules

Apriori searches for frequent item combinations. For N records, the measures are:

$$\text{support}(A \text{ and } B) = \text{count}(A \text{ and } B) / N$$

$$\text{confidence}(A \rightarrow B) = \text{support}(A \text{ and } B) / \text{support}(A)$$

$$\text{lift}(A \rightarrow B) = \text{confidence}(A \rightarrow B) / \text{support}(B)$$

To explain these measures, Table 8 shows ten **hypothetical** complaint records, each described by category, block, and priority. They are invented for explanation only.

Table 8: Hypothetical Records for Explaining Apriori

Record	Category	Block	Priority
R1	Water leakage	Block B	High
R2	Water leakage	Block B	High
R3	Electrical	Block A	Medium
R4	Wi-Fi	Block C	Low
R5	Water leakage	Block B	Critical
R6	Electrical	Block A	High
R7	Cleaning	Block C	Low
R8	Water leakage	Block A	Medium
R9	Wi-Fi	Block C	Medium
R10	Furniture	Block B	Low

Table 9: Rules Calculated from the Hypothetical Records

Rule	Support	Confidence	Lift
Water leakage → Block B	0.30	0.75	1.88
Block C → Low priority	0.20	0.67	2.22

For the first rule, 3 of the 4 water-leakage records are in Block B (confidence 0.75), and Block B appears in 4 of 10 records overall, so the lift is $0.75 / 0.40 = 1.88$: water leakage is almost twice as likely to occur in Block B as the overall share of Block B suggests. These numbers come from invented records and must not be reported as actual discoveries. Rules are to be reported only when they are calculated from the real dataset.

5.11 Modules of the System

Table 10: Proposed Modules

Module	Proposed file	Description
Web application	app.py	Page routes, request validation, and error handling
Database	database.py	Creates and queries the SQLite tables
Preprocessing	preprocessing.py	Cleaning, duplicate checks, encoding, and scaling
Training script	train_models.py	Trains and evaluates the models and saves them with Joblib
Prediction service	services/predict.py	Loads saved models and returns priority and resolution estimates
Pattern analysis	services/patterns.py	Runs K-Means and Apriori on stored records
Front end	templates/, static/	Complaint form, tables, dashboard, and charts

5.12 Technology Stack

Table 11: Technology Stack

Technology	Role
Python	Application logic and model training
Flask	Web routes and integration
HTML / CSS / JavaScript	Forms, dashboard, tables, charts
SQLite	Prototype complaint storage
Pandas / NumPy	Data cleaning and preparation
scikit-learn	Classification, regression, clustering
Joblib	Saving trained model files
WEKA-compatible ARFF	Data-mining exercises and data exchange
Render / GitHub	Hosting and source code, as described in the case study

5.13 Feature List

Table 12: Feature List

Feature	Description
Complaint management	Create, search, view, and update complaint records
Preprocessing	Prepare consistent model inputs
Priority prediction	Compare Decision Tree and Naive Bayes estimates
Resolution estimator	Provide an approximate number of days
Pattern analysis	Inspect clusters and association rules
Dashboard	Summarize status and trends

5.14 Implementation Details

The application keeps routes, storage, preprocessing, and models in separate modules so that each can be tested on its own. Model estimates are stored in separate columns from the staff-confirmed values, so the original estimate is never overwritten and the difference between estimate and decision can be studied later. Table 13 lists the proposed routes.

Table 13: Proposed Routes

Route	Method	Purpose
/	GET	Dashboard with cards and charts
/complaints/new	GET, POST	Complaint form and submission
/complaints	GET	Searchable list of complaints
/complaints/<id>/status	POST	Staff status update
/api/predict	POST	Priority and resolution estimates for valid input
/patterns	GET	Clusters and association rules
/health	GET	Simple availability check

6. VISUAL ANALYTICS AND RESULTS

6.1 Visual Analytics

A category chart helps compare complaint volumes and may highlight areas needing investigation. In an operational report, these bars should be calculated from real records grouped by category. The example values are synthetic and are not measured HostelFix results.

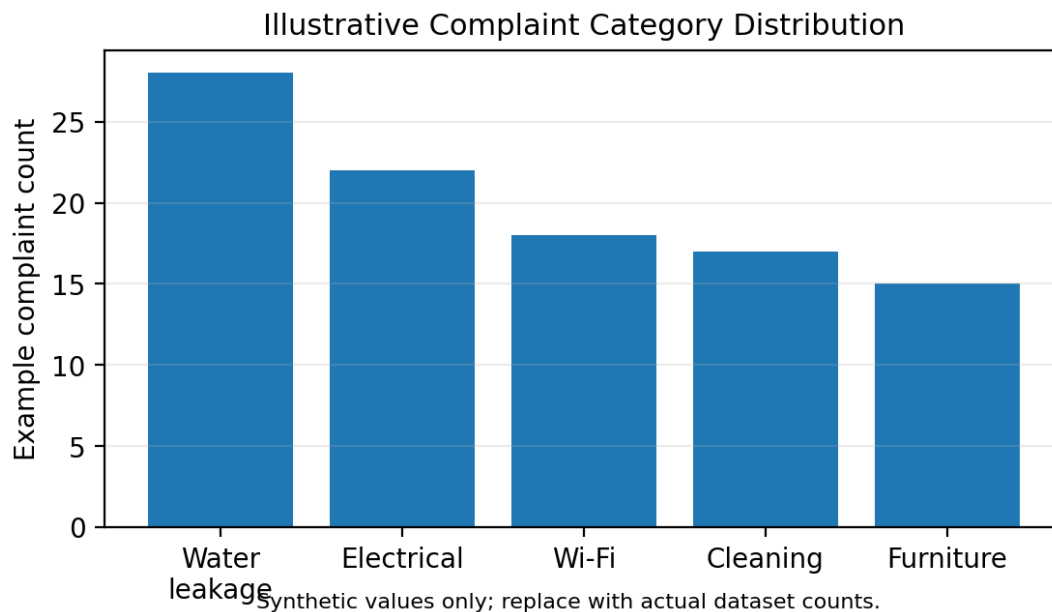


Figure 6.1: Illustrative complaint category distribution. Synthetic example only.

A resolution-time chart shows how long complaints usually take to close. A long right-hand tail may indicate repairs that depend on outside contractors or spare parts and deserve a separate look.

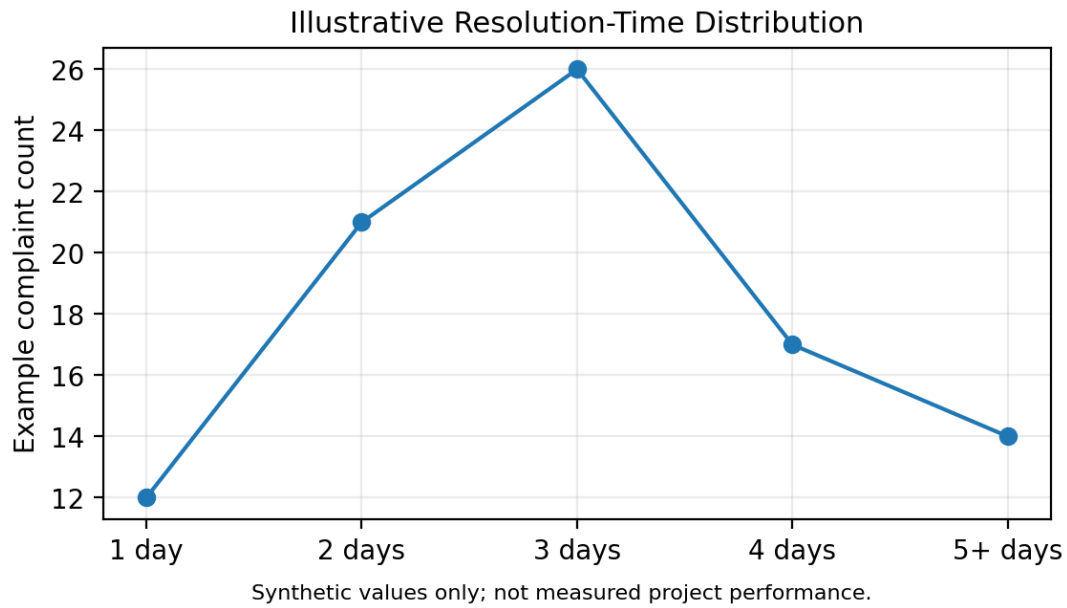


Figure 6.2: Illustrative resolution-time distribution. Synthetic example only.

6.2 Evaluation Metrics

For priority classification, each class in turn is treated as positive. With TP, TN, FP, and FN denoting true positives, true negatives, false positives, and false negatives:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN)$$

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = TP / (TP + FN)$$

$$\text{F1-score} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

$$\text{MAE} = (1 / n) \times \sum | \text{actual days} - \text{predicted days} |$$

For this problem, recall for the Critical class is especially important, because a missed critical complaint is more harmful than an extra one that staff review and downgrade.

Table 14: Evaluation Metrics

Metric	Meaning
Accuracy	Fraction of all predictions that are correct.
Precision	Among predictions for a class, how many are correct.
Recall	Among actual examples of a class, how many are identified.
F1-score	Harmonic mean of precision and recall.
MAE	Average absolute error in predicted resolution days.
Support / confidence / lift	Measures for evaluating Apriori rules.

6.3 Recommended Test Cases

Table 15: Recommended Test Cases

ID	Action	Expected result	Status
TC-01	Submit complete complaint	Record accepted and stored.	Planned
TC-02	Leave required field blank	Validation message appears.	Planned
TC-03	Use invalid severity	Rejected or handled by a documented rule.	Planned
TC-04	Request priority prediction	Priority estimate returned for valid input.	Planned
TC-05	Request resolution estimate	Numeric estimate returned if model is available.	Planned
TC-06	Open dashboard	Charts and complaint summaries display.	Planned
TC-07	Filter recurring category	Matching records can be reviewed.	Planned
TC-08	Inspect example graphs	Synthetic data is clearly labelled.	Planned

These are recommended verification cases. This report does not claim that all tests have been executed or passed.

6.4 Results and Discussion

HostelFix is presented as a design and case study. Because the source case study uses sample or synthetic data, this report does not state accuracy, precision, recall, or MAE values: any such figure computed on invented records would say nothing about real hostel conditions. What the report does establish is the pipeline: how complaints are captured, which preprocessing steps are needed, which model suits which task, and which measures should be used to judge each model.

When a real, labelled dataset is available, the models should be compared on the same records using cross-validation, class-wise precision and recall should be reported, and the final choice between Decision Tree and Naive Bayes should be justified by those numbers. Clusters and association rules should be reviewed by hostel staff before being treated as meaningful.

The example charts in this chapter are conceptual: they show how a category distribution and a resolution-time distribution would be presented, using synthetic values.

Table 16: Result Evaluation Areas

Evaluation area	Verification approach	Status
Complaint capture and validation	Test cases TC-01 to TC-03	To be executed
Priority classification	Hold-out / cross-validation on real labelled data	Needs real data
Resolution estimate	MAE on real resolution records	Needs real data
Clusters and rules	Staff review of results from real records	Needs real data
Dashboard and charts	Test cases TC-06 to TC-08	To be executed

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

HostelFix demonstrates how a web application can combine maintenance complaint management with data-mining methods. Decision Tree and Naive Bayes can estimate priority, Linear Regression can estimate resolution time, K-Means can group similar records, and Apriori can explore frequent associations. Preprocessing is essential because inconsistent or incomplete data reduces model usefulness. HostelFix should be treated as a decision-support prototype, with final decisions remaining with maintenance staff.

7.2 Expected Benefits

- Makes complaint records easier to view, search, and update.
- Helps staff identify potentially urgent complaints for review.
- Provides approximate resolution-time estimates for planning.
- Can reveal repeated categories and patterns over time.
- Connects data-mining concepts to a practical hostel scenario.

7.3 Limitations

- Predictions depend on dataset size, quality, and realism.
- Synthetic data may not represent real hostel conditions.
- Model outputs are estimates, not substitutes for staff judgment.
- Reliable production use requires secure access and persistent storage.
- Clusters and association rules can mislead if features are poorly selected.
- The example charts and hypothetical records in this report are illustrations, not results.

7.4 Future Scope

- Collect a larger, well-labelled dataset with privacy safeguards.
- Compare models through cross-validation and report class-wise metrics.
- Add alerts for high-priority or overdue complaints.
- Introduce role-based access, audit logs, and database backups.
- Show trends by hostel block, month, category, and status.
- Retrain models periodically and monitor performance.

REFERENCES

- [1] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, Morgan Kaufmann.
- [2] I. H. Witten, E. Frank, M. A. Hall, and C. J. Pal, *Data Mining: Practical Machine Learning Tools and Techniques*, Morgan Kaufmann.
- [3] Scikit-learn documentation: Decision Trees, Naive Bayes, Linear Models, and Clustering.
- [4] WEKA documentation: ARFF datasets and data-mining algorithms.
- [5] Python documentation: Python language and standard library.
- [6] Flask documentation: web application framework, <https://flask.palletsprojects.com/>
- [7] Pandas documentation: data manipulation and analysis.
- [8] NumPy documentation: numerical computing.
- [9] R. Agrawal and R. Srikant, “Fast algorithms for mining association rules,” Proceedings of the 20th International Conference on Very Large Data Bases (VLDB), 1994.
- [10] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*, Wadsworth, 1984.
- [11] J. MacQueen, “Some methods for classification and analysis of multivariate observations,” Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1967.
- [12] SQLite documentation, <https://www.sqlite.org/docs.html>
- [13] HostelFix — Smart Hostel Maintenance Analytics & Priority Prediction System, supplied project case study.

Note on figures

The workflow, architecture, entity-relationship, dashboard, decision-tree, and algorithm diagrams are conceptual. The complaint-category and resolution-time charts, the K-Means scatter plot, and the hypothetical records of Table 8 use synthetic values solely to demonstrate visual reporting and the calculation of measures; they are not measured results. Replace them with outputs generated from the actual HostelFix dataset before presenting them as project findings.

— END OF REPORT —